Complete MLOps Learning Notes — Emotion Detection Project

Introduction

These notes document the complete end-to-end Machine Learning and MLOps workflow implemented in the Emotion Detection project.

The purpose of these notes is to understand:

- Professional ML project structure
- Production-grade ML workflows
- Data pipelines
- Experiment tracking
- Model registry
- Reproducibility
- Deployment readiness
- Industry-standard MLOps practices

---

1. What is MLOps?

MLOps (Machine Learning Operations) is the practice of combining:

- Machine Learning
- DevOps
- Data Engineering
- Software Engineering

Goal:

- Build reproducible ML systems
- Automate ML workflows
- Track experiments
- Version data/models
- Deploy models reliably
- Monitor model performance

---

2. Project Architecture

```
Raw Data
   ↓
Data Preprocessing
```

```
   ↓
Dataset Creation
   ↓
Feature Engineering
   ↓
Model Training
   ↓
Model Evaluation
   ↓
Experiment Tracking
   ↓
Model Registry
   ↓
Staging / Production
```

---

3. Professional Project Structure

```
Emotaion_Detection_v5/
│
├── data/
│   ├── raw/
│   ├── processed/
│   └── feature/
│
├── models/
│   └── model.pkl
│
├── reports/
│   ├── confusion_matrix.npy
│   └── metrics.json
│
├── src/
│   ├── data/
│   ├── features/
│   ├── models/
│   └── visualization/
│
├── dvc.yaml
├── params.yaml
├── requirements.txt
├── app.log
└── README.md
```

---

4. Data Preprocessing

Purpose

Convert raw dataset into clean structured data.

Activities

- Remove null values
- Remove duplicates
- Clean text
- Train-test split
- Save train/test CSV

Example

```
train_df = train_df.fillna(0)
```

Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

---

5. Feature Engineering

What is Feature Engineering?

Converting raw text into numerical vectors.

Models cannot understand text directly.

Bag of Words (BoW)

Counts word frequency.

Example

Sentence:

```
I love AI
AI is future
```

Vocabulary:

```
[I, love, AI, is, future]
```

Vectors:

```
[1,1,1,0,0]
[0,0,1,1,1]
```

TF-IDF

Improved version of BoW.

Gives importance to unique words.

Formula

```
TF-IDF = TF × IDF
```

Code Example

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=5000)

X_train = vectorizer.fit_transform(train_text)
X_test = vectorizer.transform(test_text)
```

---

6. Model Building

Logistic Regression

Used for classification problems.

Best Parameters Used

```
LogisticRegression(
    C=1,
    solver='liblinear',
    penalty='l2',
    random_state=42,
    max_iter=1000
)
```

Meaning of Parameters

C

Inverse regularization strength.

- Small C → More regularization
- Large C → Less regularization

---

solver='liblinear'

Optimization algorithm.

Good for:

- Small datasets
- Binary classification

---

penalty='l2'

Regularization type.

Helps prevent overfitting.

Training Code

```
clf.fit(X_train, y_train)
```

Save Model

```
with open(model_path, "wb") as f:
    pickle.dump(clf, f)
```

7. Model Evaluation

Metrics Used

Accuracy

```
Correct Predictions / Total Predictions
```

Precision

Measures correctness of positive predictions.

Recall

Measures how many actual positives are identified.

F1 Score

Balance between Precision and Recall.

Formula

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

ROC-AUC

Measures classification quality.

Higher is better.

Evaluation Code

```
y_pred = clf.predict(X_test)
y_pred_proba = clf.predict_proba(X_test)[:, 1]
```

Metrics Calculation

```
metrics = {
    "accuracy": accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred),
    "recall": recall_score(y_test, y_pred),
    "f1_score": f1_score(y_test, y_pred),
    "auc_roc": roc_auc_score(y_test, y_pred_proba)
}
```

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
```

Structure:

```
TP  FP
FN  TN
```

---

8. Logging System

Why Logging?

Professional applications never use only print().

Logging helps:

- Debugging
- Error tracking
- Monitoring
- Production support

Logging Levels

```
DEBUG
INFO
WARNING
ERROR
CRITICAL
```

Logging Setup

```
logger = logging.getLogger(name)
logger.setLevel(logging.DEBUG)
```

File + Console Logging

```
console_handler = logging.StreamHandler(sys.stdout)
file_handler = logging.FileHandler("app.log")
```

---

9. DVC (Data Version Control)

What is DVC?

DVC is Git for:

- Data
- ML pipelines
- Models

Why DVC?

Git is not suitable for:

- Large datasets
- Model files
- Reproducible ML workflows

DVC solves this.

DVC Pipeline Stages

```
stages:
  model_building:
    cmd: python src/models/model_building.py
```

Run Pipeline

```
dvc repro
```

View DAG

```
dvc dag
```

DVC Concepts

deps

Dependencies.

outs

Generated outputs.

params

Hyperparameters from params.yaml.

---

10. params.yaml

Centralized configuration file.

Example

```
model_building:
  C: 1
  solver: liblinear
  penalty: l2
```

Benefits

- Easy experimentation
- Centralized config
- DVC tracking
- Reproducibility

---

11. MLflow

What is MLflow?

ML lifecycle management platform.

Tracks:

- Experiments
- Metrics
- Parameters
- Models
- Artifacts

MLflow Tracking URI

```
mlflow.set_tracking_uri(
    "https://dagshub.com/..."
)
```

Start Run

```
with mlflow.start_run():
```

Log Parameters

```
mlflow.log_param("model_type", "LogisticRegression")
```

Log Metrics

```
mlflow.log_metric("accuracy", accuracy)
```

Log Model

```
mlflow.sklearn.log_model(
    clf,
    artifact_path="model"
)
```

Log Artifacts

```
mlflow.log_artifact("reports/confusion_matrix.npy")
```

---

12. DagsHub

What is DagsHub?

Platform for:

- ML collaboration
- MLflow hosting
- DVC remote storage
- Experiment tracking

Initialization

```
import dagshub

dagshub.init(
    repo_owner="dinesh008luck",
    repo_name="Emotaion_Detection_v5",
    mlflow=True,
)
```

Benefits

- Remote experiment tracking
- Centralized artifacts
- Team collaboration
- Git integration

---

13. Model Registry

What is Model Registry?

Central storage for models.

Tracks:

- Versions
- Stages
- Metadata
- Lifecycle

Stages

```
None
Staging
Production
Archived
```

Register Model

```
mlflow.register_model(
    model_uri,
    model_name
)
```

Transition Stage

```
client.transition_model_version_stage(
    name=model_name,
    version=version,
    stage="Staging"
)
```

---

14. Common Errors Faced

ModuleNotFoundError: src

Reason

Running Python file directly.

Fix

```
python -m src.models.model_evaluation
```

DVC Output Already Tracked by Git

Fix

```
git rm --cached models/model.pkl
```

Missing params.yaml Keys

Reason

Old DVC references.

Fix

Update dvc.yaml params section.

Overlapping DVC Outputs

Reason

Two stages tracking same directory.

Fix

Track separate files instead.

---

15. Production-Level Best Practices

Use Modular Structure

Separate:

- Data
- Features
- Models
- Visualization

Avoid Hardcoding

Use:

- params.yaml
- environment variables

Always Use Logging

Avoid excessive print().

Keep Pipelines Reproducible

Use:

- DVC
- Git
- MLflow

Track Everything

Track:

- Data
- Metrics
- Models
- Parameters
- Artifacts

---

16. What is Next?

Immediate Next Steps

Docker

Containerize application.

FastAPI

Create prediction API.

CI/CD

Automate training/testing/deployment.

Cloud Deployment

Deploy on:

- AWS
- Azure
- GCP

Monitoring

Track:

- Drift
- Accuracy
- Failures
- Latency

---

17. Interview Questions

What is difference between ML and MLOps?

ML focuses on model building.

MLOps focuses on:

- automation
- deployment
- monitoring
- reproducibility

Why use DVC?

For:

- data versioning
- reproducible pipelines
- experiment management

Why MLflow?

For:

- experiment tracking
- model registry
- artifact management

Why Logistic Regression?

Simple, fast, interpretable baseline model for text classification.

---

18. Final Understanding

This project demonstrates:

- Real-world ML engineering
- End-to-end ML workflow
- Production-grade practices
- Model lifecycle management
- Experiment tracking
- Reproducibility
- Professional project architecture

Important Commands Cheat Sheet

Run Pipeline

```
dvc repro
```

Show DAG

```
dvc dag
```

Push DVC Data

```
dvc push
```

Pull DVC Data

```
dvc pull
```

Git Commit

```
git add .
git commit -m "message"
```

Run Model Evaluation

```
python -m src.models.model_evaluation
```

Run Model Registration

```
python -m src.models.model_registration
```

Final Goal

Convert ML notebooks into:

- scalable systems
- reproducible pipelines

- production-ready ML applications
- deployable services

That is the real transition from:

```
Data Scientist → ML Engineer → MLOps Engineer
```